Name: Yash Gawade

Roll no: **46**

Assignment no.-6

Import Libraries

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score,
confusion_matrix
```

```
from google.colab import files
uploaded = files.upload()
```

```
<IPython.core.display.HTML object>
```

```
Saving diabetes.csv to diabetes.csv
```

Load Dataset

```
import pandas as pd

df = pd.read_csv('diabetes.csv')
df.head()

df.shape
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
```

```
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

{"summary":"{\n \"name\": \"df\",\n \"rows\": 8,\n \"fields\": [\n {\n \"column\": \"Pregnancies\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 269.85223453356366,\n \"min\": 0.0,\n \"max\": 768.0,\n \"num_unique_values\": 8,\n \"samples\": [\n 3.8450520833333335,\n 3.0,\n 768.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Glucose\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 243.73802348295857,\n \"min\": 0.0,\n \"max\": 768.0,\n \"num_unique_values\": 8,\n \"samples\": [\n 120.89453125,\n 117.0,\n 768.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"BloodPressure\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 252.85250535810619,\n \"min\": 0.0,\n \"max\": 768.0,\n \"num_unique_values\": 8,\n \"samples\": [\n 69.10546875,\n 72.0,\n 768.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"SkinThickness\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 263.7684730531098,\n \"min\": 0.0,\n \"max\": 768.0,\n \"num_unique_values\": 7,\n \"samples\": [\n 768.0,\n 20.536458333333332,\n 32.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Insulin\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 350.26059167945886,\n \"min\": 0.0,\n \"max\": 846.0,\n \"num_unique_values\": 7,\n \"samples\": [\n 768.0,\n 79.79947916666667,\n 127.25\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"BMI\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 262.05117817552093,\n \"min\": 0.0,\n \"max\": 768.0,\n \"num_unique_values\": 8,\n \"samples\": [\n 31.992578124999998,\n 32.0,\n 768.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"DiabetesPedigreeFunction\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 271.3005221658502,\n \"min\": 0.078,\n \"max\": 768.0,\n \"num_unique_values\": 8,\n \"samples\": [\n 0.47187630208333325,\n 0.3725,\n 768.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Age\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 260.1941178528413,\n \"min\": 11.76023154067868,\n \"max\": 768.0,\n \"num_unique_values\": 8,\n \"samples\": [\n 33.240885416666664,\n 29.0,\n 768.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n

{\n   \"column\": \"Outcome\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 271.3865920388932,\n   \"min\":
0.0,\n   \"max\": 768.0,\n   \"num_unique_values\": 5,\n
\"samples\": [\n   0.3489583333333333,\n   1.0,\n
0.4769513772427971\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

Data Cleaning

```
cols = ['Glucose','BloodPressure','SkinThickness','Insulin','BMI']
df[cols] = df[cols].replace(0, np.nan)
```

```
df.fillna(df.median(), inplace=True)
```

Fill Missing Values

```
df.fillna(df.median(), inplace=True)
df.isnull().sum()
```

```
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

Frequency Distribution

```
df['Outcome'].value_counts()
```

```
Outcome
0    500
1    268
Name: count, dtype: int64
```

Statistical Measures

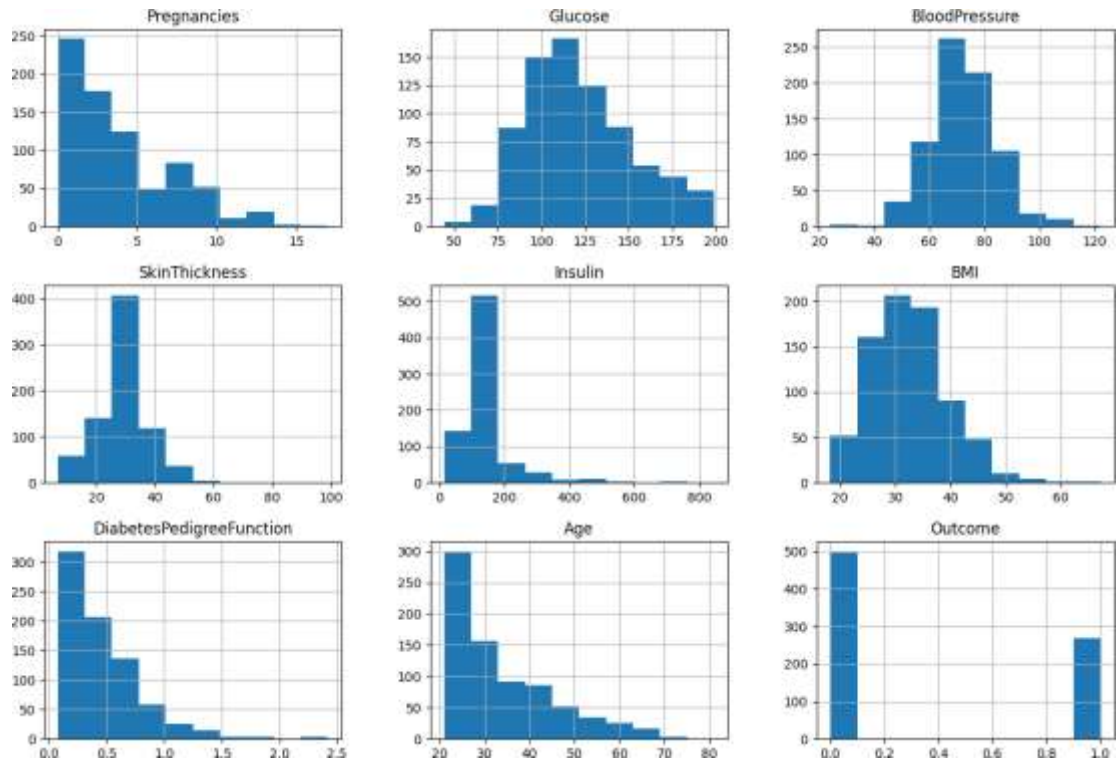
```
univariate = pd.DataFrame({
    'Mean': df.mean(),
    'Median': df.median(),
    'Mode': df.mode().iloc[0],
    'Variance': df.var(),
    'Std Dev': df.std(),
    'Skewness': df.skew(),
    'Kurtosis': df.kurt()
})
```

univariate

{"summary":"{\n \"name\": \"univariate\",\n \"rows\": 9,\n \"fields\": [\n {\n \"column\": \"Mean\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 52.324805149927464,\n \"min\": 0.3489583333333333,\n \"max\": 140.671875,\n \"num_unique_values\": 9,\n \"samples\": [\n 33.240885416666664,\n 121.65625,\n 32.45520833333333\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Median\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 48.433085504929146,\n \"min\": 0.0,\n \"max\": 125.0,\n \"num_unique_values\": 8,\n \"samples\": [\n 117.0,\n 32.3,\n 3.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Mode\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 45.945747555616556,\n \"min\": 0.0,\n \"max\": 125.0,\n \"num_unique_values\": 9,\n \"samples\": [\n 22.0,\n 99.0,\n 32.3\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Variance\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 2448.6312632019653,\n \"min\": 0.10977863787313938,\n \"max\": 7462.033001955671,\n \"num_unique_values\": 9,\n \"samples\": [\n 138.30304589037365,\n 926.4892438070405,\n 47.26805627987832\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Std Dev\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 27.263407988701612,\n \"min\": 0.33132859501277484,\n \"max\": 86.383059693181,\n \"num_unique_values\": 9,\n \"samples\": [\n 11.76023154067868,\n 30.43828582241517,\n 6.875176818080996\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Skewness\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.9797769059153865,\n \"min\": 0.1418850201030565,\n \"max\": 3.380019128212173,\n \"num_unique_values\": 9,\n \"samples\": [\n 1.1295967011444805,\n 0.5355873034111183,\n 0.599232520287425\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Kurtosis\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 5.487769090233668,\n \"min\": -1.600929755156027,\n \"max\": 16.232455064101348,\n \"num_unique_values\": 9,\n \"samples\": [\n 0.6431588885398942,\n -0.2578474160780089,\n 0.9202217134186936\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe","variable_name":"univariate"}

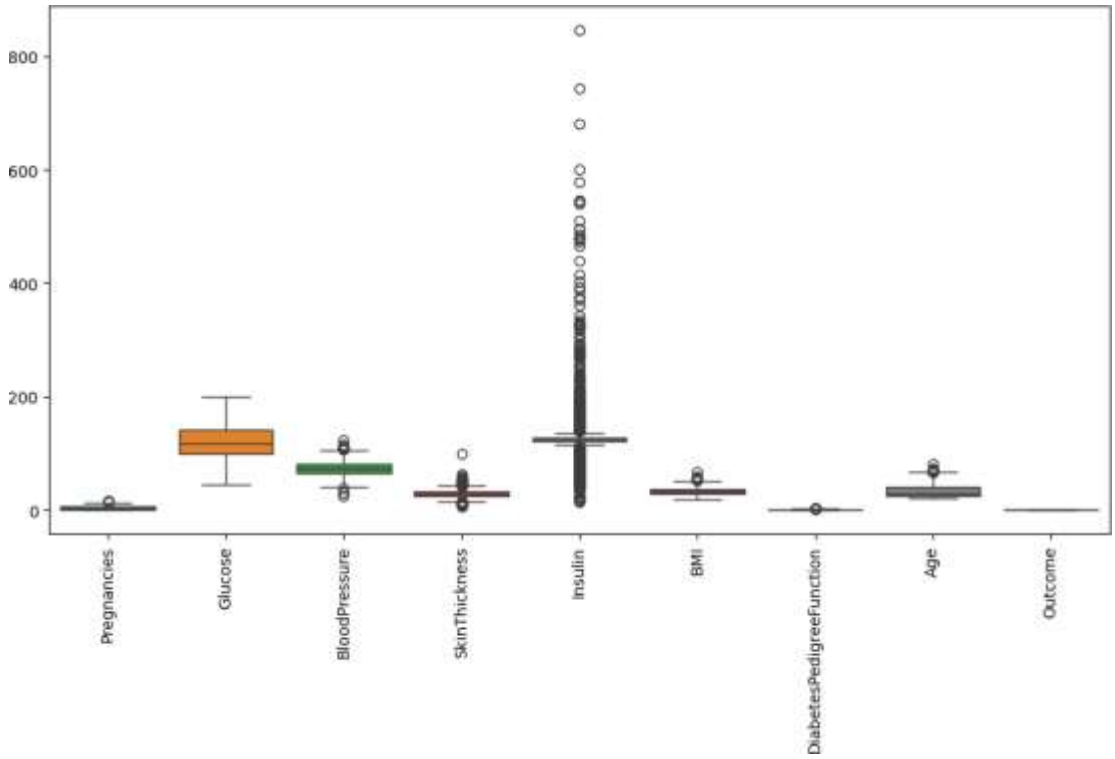
Histogram

```
df.hist(figsize=(15,10))
plt.show()
```



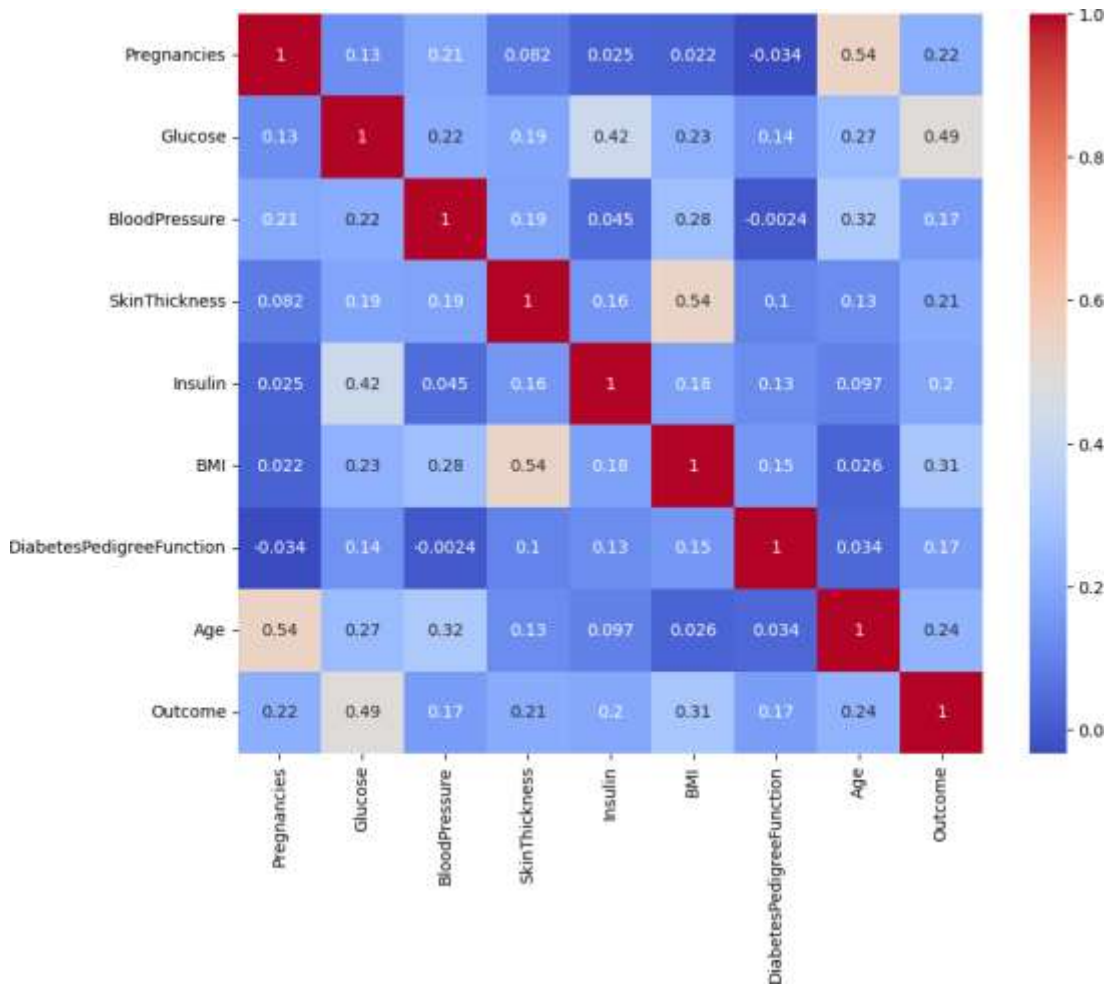
Boxplot

```
plt.figure(figsize=(12,6))
sns.boxplot(data=df)
plt.xticks(rotation=90)
plt.show()
```



PART2-BIVARIATE ANALYSIS

```
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
```



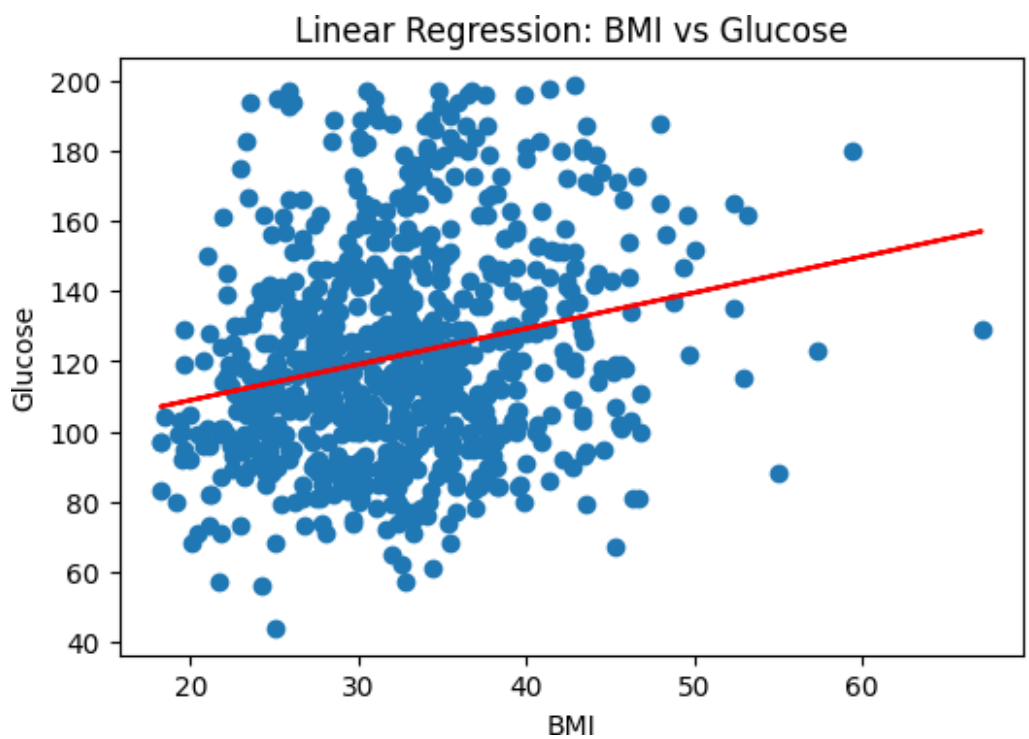
Linear Regression (BMI vs Glucose)

```
X = df[['BMI']]
y = df['Glucose']

lr = LinearRegression()
lr.fit(X,y)

print("Slope:", lr.coef_[0])
print("Intercept:", lr.intercept_)
```

```
Slope: 1.0229150467093613
Intercept: 88.45732905174629
```

```
# recreate X and y again (VERY IMPORTANT)
X = df[['BMI']]
y = df['Glucose']

# train model again
from sklearn.linear_model import LinearRegression
```

```
lr = LinearRegression()
lr.fit(X, y)

# plot
plt.figure(figsize=(6,4))
plt.scatter(X['BMI'], y)
plt.plot(X['BMI'], lr.predict(X), color='red')
plt.xlabel("BMI")
plt.ylabel("Glucose")
plt.title("Linear Regression: BMI vs Glucose")
plt.show()
```



Logistic Regression (Predict Diabetes)

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
from sklearn.preprocessing import StandardScaler

X = df.drop('Outcome', axis=1)
y = df['Outcome']

X_train, X_test, y_train, y_test =
train_test_split(X,y,test_size=0.2,random_state=42)

model = LogisticRegression(max_iter=2000)
```

```
model.fit(X_train,y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test,pred))
print(confusion_matrix(y_test,pred))
print(classification_report(y_test,pred))
```

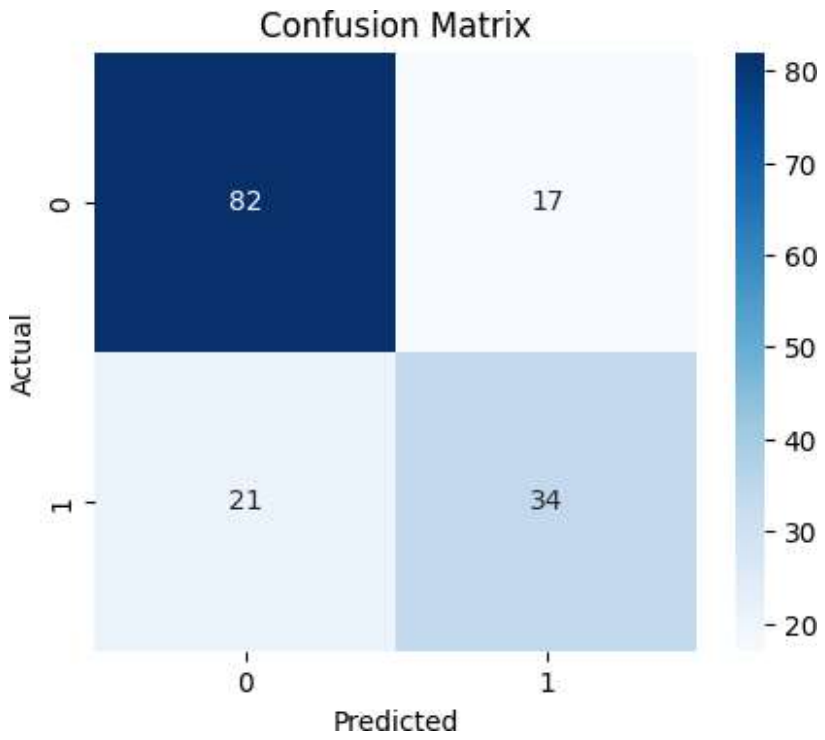
```
Accuracy: 0.7532467532467533
[[82 17]
 [21 34]]
              precision    recall  f1-score   support

           0       0.80      0.83      0.81        99
           1       0.67      0.62      0.64        55

    accuracy                           0.75       154
   macro avg       0.73      0.72      0.73       154
weighted avg       0.75      0.75      0.75       154
```

MULTIPLE REGRESSION

```
X = df[['Pregnancies','BloodPressure','SkinThickness','Insulin','BMI','Age']]
y = df['Glucose']

multi = LinearRegression()
multi.fit(X,y)

print("Coefficients:", multi.coef_)
print("Intercept:", multi.intercept_)
```

```
Coefficients: [-0.14491309  0.25849064  0.1003194   0.1305834   0.50849502
0.516832  ]
Intercept: 48.52937876789389
```

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, pred)

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



Link:
https://colab.research.google.com/drive/1X7IQG94Rl7XnODSvmMV6aoTXQidCBPnM?usp=sharing